

Adamrit ↔ Tally Sync Architecture & the Accounting Module

A complete reference for the bidirectional real-time integration between the Adamrit hospital management system and TallyPrime, the auto-posting double-entry ledger, the database schema, the XML protocol, and operational sync behaviour.

Double-Entry Bookkeeping

TallyPrime XML

Event-Driven Push

Postgres Triggers

Vercel Proxy

Supabase

Contents

- 1.** Executive Summary
- 2.** The Three Data Planes
- 3.** System Topology & Deployment
- 4.** Accounting Module Deep Dive (Plane A)
 - › Chart of Accounts & Master Data
 - › The Voucher / Journal-Entry Model
 - › Trigger-Based Auto-Posting
 - › Hospital Events → Accounting Matrix
- 9.** Forward Sync — Adamrit → Tally
 - › The Auto-Push Service
 - › The Vercel Tally Proxy
 - › Domain → Voucher Mapping
 - › Retry Queue & Reliability
- 14.** Reverse Sync — Tally → Adamrit
 - › Server-Side Collection Pulls
 - › Client-Side Receipt Matcher
 - › Reports & Caching
- 18.** The TallyPrime XML Protocol
 - › Import Envelopes
 - › Export / Collection Envelopes
 - › Sign & Date Conventions
- 22.** Database Schema Reference
- 23.** The Mapping Layer
- 24.** Sync Modes, Scheduling & Triggers
- 25.** Reconciliation & Reporting Surfaces
- 26.** Error Handling, Idempotency & Security
- 27.** Known Gaps & Architectural Caveats
- 28.** Appendix — File & Code Reference Index

1. Executive Summary

Adamrit is a Supabase-backed hospital management application (Next.js / React) that maintains its own double-entry books *and* keeps a live, two-way synchronisation with one or more TallyPrime company files running on the hospital LAN. This document describes both halves of that system in full.

The integration is best understood as a **real-time, event-driven push** (Adamrit → Tally) paired with a **scheduled, multi-strategy pull** (Tally → Adamrit), all brokered by a single Vercel serverless proxy that speaks TallyPrime's XML-over-HTTP protocol. Internally, hospital business events are converted into balanced vouchers by Postgres triggers, so the accounting books populate themselves the moment a payment is recorded — with no manual entry required for receipts.

AT A GLANCE

Forward sync (Adamrit → Tally) is fire-and-forget: when a bill is saved, an advance payment posted, or a pharmacy sale rung up, a helper builds a Tally XML voucher, posts it through the proxy to the Tally gateway (default `http://<host>:9000`), mirrors the result locally, and — if Tally is unreachable — enqueues it for retry.

Reverse sync (Tally → Adamrit) pulls ledgers, vouchers, stock, reports and GST data back into Supabase mirror tables, auto-matches Tally receipts against patient visits, and powers bank reconciliation, outstanding tracking and financial reporting.

Direction	Mode	Mechanism	Trigger
ADAMRIT → TALLY	Real-time / event-driven	<code>tally-auto-push.ts</code> → <code>/api/tally-proxy</code>	Inline at business-record save (bill, payment, sale)
FORWARD RETRY	Periodic, browser-side	<code>processRetryQueue()</code> via <code>TallyRetryQueue.tsx</code> 5-min interval	While the Retry Queue tab is open
TALLY → ADAMRIT	Scheduled, browser-side	<code>TallyDashboard</code> / <code>TallyAutoSync</code> 30-s scheduler	Configurable 1–24 h auto-sync, or manual
REVERSE STANDALONE	OS cron	<code>scripts/tally-direct-sync.js</code> (HTTP) or <code>sync-tally-export.js</code> (file)	Windows Task Scheduler / cron

ARCHITECTURAL CAVEAT

There is **no server-side worker** for the forward retry queue or the reverse auto-sync scheduler. Both run as browser `setInterval` loops, so they only make progress while a user has the relevant Tally page open in a browser tab.

How to read this document

Sections 4–5 cover the accounting module itself (where financial truth lives). Sections 6–8 cover the two sync directions and the XML protocol. Sections 9–11 are reference material (schema, mappings, scheduling, security). Every claim is anchored to a `file:line` reference in the codebase; the full index is in §13.

2. The Three Data Planes

A single point of confusion dominates this codebase: financial data lives in **three parallel, overlapping table families**. Understanding which plane you are reading is essential before touching any accounting or Tally code.

Plane	Tables	Purpose	Authoritative?
A Operational ledger	<code>chart_of_accounts</code> , <code>voucher_types</code> , <code>vouchers</code> , <code>voucher_entries</code> , <code>patient_ledgers</code> , <code>outstanding_invoices</code> , <code>payment_transactions</code> , <code>payment_allocations</code> , <code>aging_snapshots</code> , <code>daily_balances</code> , <code>cost_centres</code> , <code>voucher_edit_log</code>	The hospital's own double-entry books. Hospital events auto-post vouchers here via DB triggers . All financial truth lives here.	Yes — source of truth for Adamrit's books
B Tally-style masters	<code>ledger_groups</code> , <code>ledgers</code> , <code>payment_vouchers</code> , <code>payment_voucher_lines</code> , <code>cost_centres</code>	TallyPrime-compatible master UI (groups/ledgers with mailing, GST, banking fields). Used for the Tally look-alike screens and file import/export round-tripping.	Partial — drives the master UI
C Tally live-sync mirror	<code>tally_config</code> , <code>tally_ledgers</code> , <code>tally_groups</code> , <code>tally_vouchers</code> , <code>tally_stock_items</code> , <code>tally_reports</code> , <code>tally_gst_data</code> , <code>tally_sync_log</code> , <code>tally_push_queue</code> , <code>tally_ledger_mapping</code> , <code>tally_bank_statements</code> , <code>payment_obligation_ledgers</code>	Mirror of an external Tally company's books, populated by the Tally gateway proxy. Separate from Plane A.	Sync only — reflects external Tally

THE BRIDGE BETWEEN PLANES

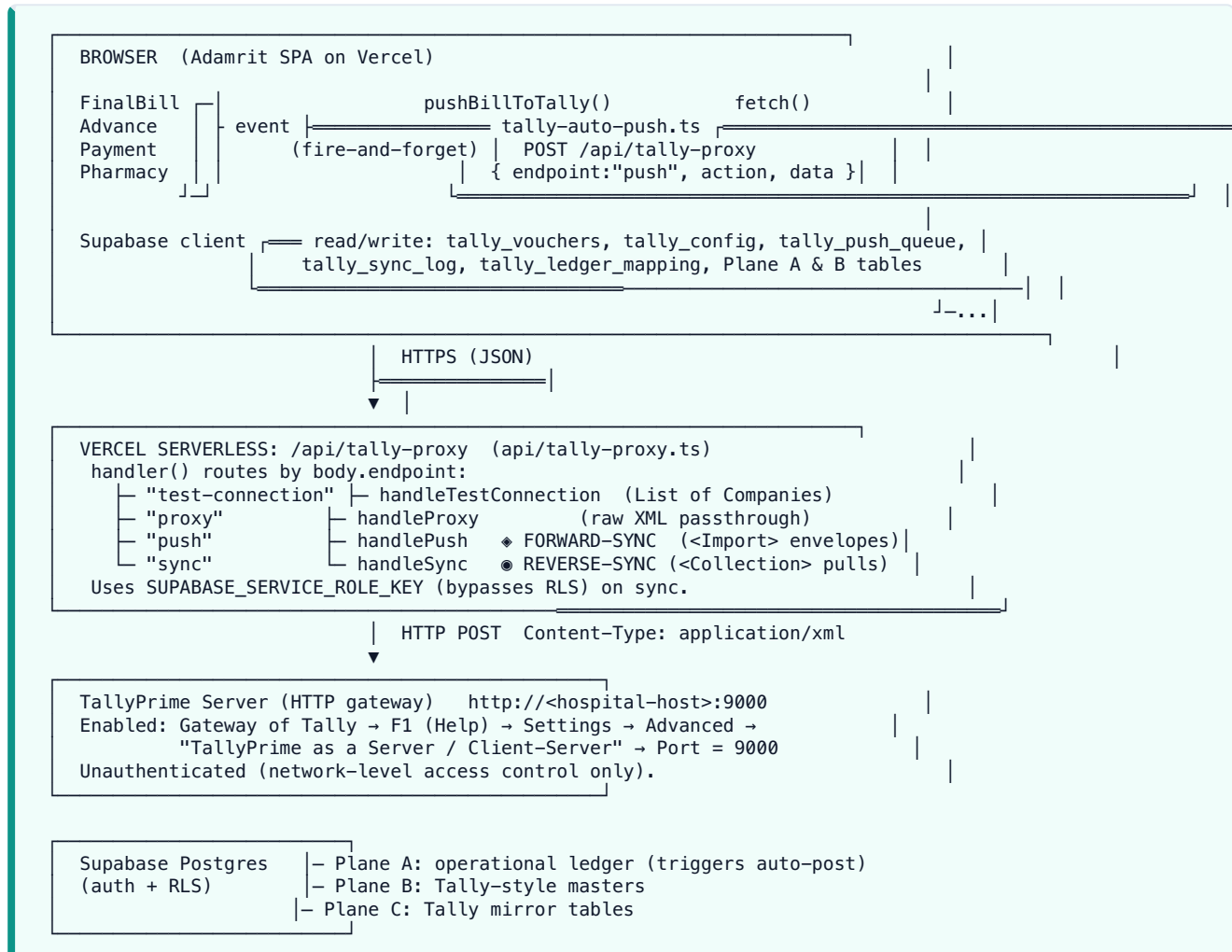
Plane A and Plane C share **names but not IDs**. The voucher exporter reads Plane A's `voucher_entries` and emits Tally XML using `chart_of_accounts.account_name` as the ledger name — so ledger names must match exactly across planes for import to succeed. The only explicit ID link is `payment_obligation_ledgers`, which joins a Plane-A payable obligation to Plane-C Tally company ledgers for the director's cross-company outstanding view.

RECONSTRUCTION WARNING

The `ledgers` and `ledger_groups` base tables (Plane B) were created **out-of-band** — there is no tracked migration defining their original columns, only `ALTER TABLE ... ADD COLUMN IF NOT EXISTS` extensions. Recreating the database from migrations alone would **not** produce these tables.

3. System Topology & Deployment

The frontend is a Next.js SPA deployed on Vercel, backed by Supabase (Postgres + RLS). The TallyPrime gateway runs on the hospital LAN and is reachable only from inside the network — hence the proxy + LAN-side design.



Why a proxy at all?

The Tally gateway is an unauthenticated XML endpoint that usually binds to a LAN/private IP. The browser cannot reach it directly (CORS + private-network restrictions), and Adamrit runs on Vercel. The serverless proxy `api/tally-proxy.ts` is the single component that performs the actual HTTP call to Tally, translates JSON ↔ XML, and (on the reverse path) bulk-upserts into Supabase using the service-role key.

REACHABILITY GOTCHA

Because the Vercel function calls the hospital-LAN Tally gateway, the hospital must expose port **9000** to the internet (port-forward / tunnel). The proxy's `buildTallyReachabilityError()` produces a human-readable message explaining the firewall / port-forwarding requirements when a connection times out. `api/tally-proxy.ts:42-47`

4. Accounting Module Deep Dive (Plane A)

The accounting module reproduces TallyPrime's look-and-feel inside Adamrit. Its shell `AccountingPage.tsx` hosts 32 destinations organised exactly like Tally's Gateway of Tally — but behind the chrome is a genuine double-entry ledger driven by Postgres triggers.

4.1 Module shell & navigation

`AccountingPage.tsx` (`src/components/accounting/AccountingPage.tsx`) is the master layout: a collapsible left icon-rail, a `TallyTopBar` chrome, and a scrollable canvas. It manages tab routing and Tally's classic drill-down stack:

```
Report ▶ Group Summary ▶ Ledger View ▶ Voucher (Alter mode)
      (drillGroup)       (drillLedgerId)  (alterVoucherId)
```

The `/tally` route (`TallyIntegration.tsx`) renders the same shell with `initialTab="tally-live"` — the live Tally suite is deliberately consolidated inside the accounting chrome. Esc returns to the Gateway; Tally hot-letter keys (D, C, etc.) are honoured. The 12-screen live Tally suite (`TallyPage`) is `React.lazy`-loaded to keep the initial bundle lean.

4.2 Chart of Accounts & master data

The canonical chart of accounts is seeded by `20250613000000_insert_accounting_default_data.sql`. It is a self-referential tree on `chart_of_accounts` (each row has `parent_account_id`).

```
-- chart_of_accounts shape (migration 20250612230000)
id UUID PK
account_code TEXT UNIQUE          -- e.g. '1110'
account_name TEXT                 -- 'Cash in Hand'
account_type TEXT CHECK IN (13 values: ASSETS, LIABILITIES, EQUITY,
                             INCOME, EXPENSES, CURRENT_ASSETS, FIXED_ASSETS,
                             CURRENT_LIABILITIES, LONG_TERM_LIABILITIES,
                             DIRECT_INCOME, INDIRECT_INCOME, DIRECT_EXPENSES, INDIRECT_EXPENSES)
parent_account_id UUID FK -> chart_of_accounts(id) -- self-ref tree
account_group, is_active BOOLEAN
opening_balance NUMERIC(15,2)
opening_balance_type TEXT CHECK IN ('DR','CR')
```

Code	Account	Type	Group	Used for
1000	ASSETS	PRIMARY	—	Balance-sheet header (DR)
1100	Current Assets	CURRENT_ASSETS	1000	Liquidity bucket
1110	Cash in Hand	CASH	1100	Default debit target for cash payments
1120	Cash at Bank	BANK	1100	Bank grouping
1121	STATE BANK OF INDIA (DRM)	BANK	1120	Leaf bank account
1122	SARASWAT BANK	BANK	1120	Leaf bank account
1124	Canara Bank HOPE PHARMACY	BANK	1120	Pharmacy non-cash payments
1400	Patient Debtors	DEBTORS	1100	Per-patient debtor sub-ledgers
1600	Fixed Assets	FIXED_ASSETS	1000	Equipment, furniture, computers
2000	LIABILITIES	PRIMARY	—	Balance-sheet header (CR)
2110	Patient Advance	ADVANCE	2100	Credit target for advance receipts
2120	Other Creditors	CREDITORS	2100	Suppliers / payables
2140	TDS Payable	—	2100	TDS deducted at payment (manual entry)
3000	CAPITAL ACCOUNT	PRIMARY	—	Owner's capital, retained earnings (CR)
4000	INCOME	PRIMARY	—	Income header (CR)
4110– 4150	Consultation / Procedure / Surgery / Investigation / Medicine Sales	DIRECT_INCOME	4100	Service-line revenue leaves
4160	Hospital Services Income	DIRECT_INCOME	4100	Auto-created credit target for receipts
5000	EXPENSES	PRIMARY	—	Expense header (DR)
5110– 5290	Medicine Purchase, Supplies, Salary, Rent, Utilities...	EXPENSES	5100/5200	Expense leaves

Default voucher types seeded: **RV** Receipt, **PV** Payment, **JV** Journal, **SV** Sales, **CN** Credit Note, **DN** Debit Note, **CV** Contra, plus sales sub-types **PF/PC/MC**. Each carries `numbering_method` (AUTO/MANUAL), `prefix`, `current_number` for sequence generation.

4.3 The voucher / journal-entry model

Every transaction is a **header row** in `vouchers` plus one or more balanced **line rows** in `voucher_entries`.

```

-- vouchers (header)
id, voucher_number, voucher_type_id FK->voucher_types,
voucher_date, reference_number/date, narration,
total_amount NUMERIC(15,2),          -- = SUM(debits)
patient_id FK->patients, bill_id FK->bills,
company_id (multi-company), is_optional BOOLEAN,
status CHECK IN ('PENDING','AUTHORISED','CANCELLED'),
created_by / authorised_by / last_modified_by -> auth.users

-- voucher_entries (lines)
id, voucher_id FK->vouchers (ON DELETE CASCADE),
account_id FK->chart_of_accounts,
patient_ledger_id FK->patient_ledgers, -- sub-ledger pointer (nullable)
cost_centre_id FK->cost_centres,       -- departmental allocation (nullable)
bill_ref TEXT,                         -- bill-wise outstanding ref
debit_amount, credit_amount NUMERIC(15,2),
narration, entry_order INT
CHECK (NOT (debit_amount > 0 AND credit_amount > 0)) -- a line is Dr XOR Cr

```

BALANCING & BALANCES

Posted vouchers must satisfy $|\Sigma \text{debit} - \Sigma \text{credit}| \leq 0.01$ (validated client-side in `VoucherEntry.tsx:661`). There is **no stored running balance** on `chart_of_accounts` — every report recomputes balances live by aggregating `voucher_entries` joined to authorised vouchers. Opening balances come from `chart_of_accounts.opening_balance + opening_balance_type`. The only stored running balance is `patient_ledgers.current_balance`, maintained by a trigger.

4.4 Trigger-based auto-posting

The single most important component bridging hospital operations to the ledger is the PL/pgSQL trigger function `create_receipt_voucher_for_payment()`, defined most recently in migration `20260704200000_receipt_voucher_proper_ledgers.sql (:1-337)`. It is attached to three tables and fires **AFTER INSERT**.

Table	Trigger	Posts when
<code>advance_payment</code>	<code>create_receipt_voucher_on_advance_payment</code>	IPD advance taken
<code>final_payments / ipd_final_bill_payments</code>	<code>create_receipt_voucher_on_final_bill_payment</code>	Discharge payment
<code>patient_payment_transactions</code>	<code>trg_patient_payment_create_voucher</code>	OPD / pharmacy / physio / direct-sale payment

The posting algorithm

- Extract** amount + payment mode from the inserted row (column names differ per table — `advance_amount` vs `amount`).
- Skip** zero/negative amounts (guard added after junk â€ 0.00 vouchers appeared).
- Resolve the DEBIT account** (money in) via a cascading decision:
 - CASH** → **1110** Cash in Hand (unless `remarks` mentions `sbi/saraswat` — legacy keyword routing).
 - ONLINE/UPI/NEFT/RTGS/CARD/CHEQUE/DD** → resolved via `bank_account_id`, then `bank_account_name`, then `remarks` keyword.

- **Pharmacy routing:** a non-cash `patient_payment_transactions` row with `payment_source` IN ('PHARMACY', 'DIRECT_SALE') and no bank set is forced to 1124 Canara Bank HOPE PHARMACY.
 - Final fallback → 1110.
4. **Resolve the CREDIT account:** `advance_payment` → 2110 Patient Advance (a liability); everything else → 4160 Hospital Services Income (auto-created if missing).
 5. **Generate voucher number** via `generate_voucher_number(voucher_type_code)` (reads & increments `voucher_types.current_number`).
 6. **Insert header** (`status='AUTHORISED'`, auto-authorized) with `reference_number` = visit_id or payment id.
 7. **Insert two balanced lines:** Dr Bank/Cash for full amount; Cr Income/Patient-Advance for full amount.

RESULT

Money-in events post themselves: Dr Bank/Cash, Cr Income/Advance. The books populate automatically the moment any payment row is inserted. Money-out and charge-raising events do **not** auto-post — they require manual Voucher Entry.

Secondary pharmacy trigger

`create_voucher_for_pharmacy_credit_payment()` in 20260325000002 fires on `pharmacy_credit_payments`. Same Dr-Bank / Cr-INCOME pattern; routes UPI/CARD/ONLINE/NEFT/RTGS to 1124, CASH to 1110, and skips `payment_method='CREDIT'` rows (those are credit extensions, not payments).

Balance trigger on patient ledgers

`update_patient_ledger_balance()` AFTER INSERT/UPDATE/DELETE ON `voucher_entries` recomputes the affected `patient_ledgers.current_balance` as `SUM(debit_amount - credit_amount)` and flips the DR/CR sign. migration 20250612230000:226-259

4.5 Hospital events → accounting matrix

Hospital event	Source table	Auto?	Dr	Cr	Voucher
IPD advance	<code>advance_payment</code>	YES	Bank/Cash	2110	Receipt
IPD discharge payment	<code>final_payments</code>	YES	Bank/Cash	4160	Receipt
OPD consultation payment	<code>patient_payment_transactions</code>	YES	Bank/Cash	4160	Receipt
Pharmacy counter sale (credit pay)	<code>pharmacy_credit_payments</code>	YES	1124/Cash	4000	Receipt
Pharmacy bill payment (patient)	<code>patient_payment_transactions</code>	YES	1124/Cash	4160	Receipt
Physiotherapy / direct sale	<code>patient_payment_transactions</code>	YES	Bank/Cash	4160	Receipt
Lab investigation charge	(lab orders)	MANUAL	1400	4140	Sales/ Journal
Surgery / procedure charge	(bill prep)	MANUAL	1400	4120/4130	Sales/ Journal
Expense / bill payment	<code>payment_vouchers</code>	MANUAL	Expense	Bank/Cash + 2140	Payment
Payroll / salary	(staff module)	MANUAL	5210	Bank/Cash	Payment
ESIC / insurance	—	MANUAL	Receivable	Bank/Cash	Journal/ Payment
Purchase (medicines/supplies)	(GRN)	MANUAL	5110/5120/ stock	2120	Purchase/ Journal
Bank/cash transfers	—	MANUAL	Bank/Cash	Bank/Cash	Contra

IMPORTANT GAP

The only *automated* postings are **receipts** (money in). Revenue recognition (raising charges against patient debtors), expenses, purchases, payroll, and ESIC/insurance are entered manually through Voucher Entry. The `outstanding_invoices` table exists to track receivables but is **not** auto-populated by a bill-creation trigger.

Manual Voucher Entry specifics

`VoucherEntry.tsx` handles every non-triggered transaction. `buildEntries()` (:575-638) constructs lines in two modes: **single-account** (PAYMENT/RECEIPT/CONTRA — one "Account" ledger offset by multiple "Particulars") or **journal** (free-form Dr/Cr). TDS at payment auto-generates a Cr row to 2140 at section rates (194J 10%, 194C 2%/1%, 194I 10%, 194H 5%). Alter mode deletes-then-re-inserts entries but preserves the voucher number. Receipts ≥ ₹10,000 fire `sendPaymentAlert()`.

5. Forward Sync — Adamrit → Tally

Forward sync is a **fire-and-forget, event-driven push**. When a business event happens, the relevant page calls a named helper in `tally-auto-push.ts`, which checks Tally is configured, resolves ledger mappings, POSTs a JSON payload to the proxy, mirrors the result locally, and enqueues a retry on failure.

5.1 The auto-push service (src/lib/tally-auto-push.ts, 830 lines)

This is the primary forward-sync layer and the one wired into all live business flows. It exposes named push functions, each following the same lifecycle:

```
pushXxxToTally(payload)
├── 1. isTallyActive()           -> SELECT tally_config WHERE is_active=true LIMIT 1
                                returns {active, serverUrl, companyName, companyId}
                                (any error -> silent skip, return)
├── 2. getLedgerMapping(companyId) -> tally_ledger_mapping (is_active=true)
                                fallback: tally_config.metadata.ledgerMapping
                                fallback: hardcoded default map
├── 3. fetch('/api/tally-proxy', {endpoint:'push', action, data})
├── 4. logPush(type, success, errors, ref, companyId) -> tally_sync_log (silent on fail)
├── success: mirrorVoucherToLocal(v) -> tally_vouchers (sync_direction:'to_tally', sync_status:'synced')
                                dedup by (company_id, voucher_number) (silent on fail)
└── failure: enqueueForRetry(type, action, payload, error, ref, companyId)
            -> tally_push_queue (status:'pending', retry_count:0, max_retries:5,
                                next_retry_at = now + 5 min) (silent on fail)
```

Push function	Action	Tally voucher	Called from
<code>pushLedgerToTally</code> (:179)	<code>create- ledger</code>	(master)	<code>PaymentVoucher.tsx:279</code>
<code>pushBillToTally</code> (:255)	<code>create- sales- voucher</code>	Sales	<code>FinalBill.tsx:16002</code>
<code>pushPaymentToTally</code> (:324)	<code>create- receipt- voucher</code>	Receipt	<code>AdvancePaymentModal.tsx:651</code>
<code>pushPharmacySaleToTally</code> (:671)	<code>create- sales- voucher</code>	Sales	<code>DirectSaleBill.tsx:595</code> , <code>PharmacyBilling.tsx:738</code> , <code>PrescriptionQueue.tsx:561</code>
<code>pushPaymentVoucherToTally</code> (:751)	<code>create- voucher (Payment)</code>	Payment	<code>PaymentVoucher.tsx:618</code>
<code>pushESICBillToTally</code> (:393)	<code>create- voucher (Sales)</code>	Sales	ESIC billing flow (API-ready)
<code>pushInsuranceBillToTally</code> (:489)	<code>create- voucher (Sales)</code>	Sales	Insurance billing flow
<code>pushInsurancePaymentToTally</code> (:582)	<code>create- voucher (Receipt)</code>	Receipt	Insurance settlement

UI-driven pushes (manual create / alter / cancel / re-push) bypass the service and POST directly to `/api/tally-proxy` from `TallyCreateVoucher.tsx:468`, `TallyVouchers.tsx:120/235`, and `TallyBillSync.tsx:68`.

5.2 The Vercel Tally proxy (api/tally-proxy.ts, 660 lines)

The **only** component that talks HTTP to Tally. Runs server-side to bypass browser CORS and reach LAN/private IPs. Its `handler()` (:620) dispatches on `body.endpoint` to one of four handlers. The forward-sync entry point is `handlePush()` (:172-285), which requires `{action, serverUrl, companyName, data}` and builds XML via a `switch(action)`:

action	XML built	Envelope ID	Lines
<code>create-ledger</code>	<code><LEDGER ACTION="Create"></code>	All Masters	:178-193
<code>create-voucher</code>	generic <code><VOUCHER ACTION="Create"></code>	Vouchers	:195-212
<code>create-sales-voucher</code>	<code><VOUCHER VCHTYPE="Sales"></code>	Vouchers	:213-228
<code>create-receipt-voucher</code>	<code><VOUCHER VCHTYPE="Receipt"></code>	Vouchers	:229-243
<code>alter-voucher</code>	<code><VOUCHER ACTION="Alter"></code>	Vouchers	:244-262
<code>cancel-voucher</code>	<code><VOUCHER ACTION="Delete"></code>	Vouchers	:263-274

Each calls `fetchFromTally()` (:99-124) — an `AbortController`-guarded POST of `application/xml` — then `parseResponse()` (:78-97).

LENIENT SUCCESS DETECTION

The server treats a response as successful when `errors.length === 0`, even if `CREATED=0/ALTERED=0` (:89-92) — deliberately lenient because TallyPrime sometimes returns zero counts even on a successful import. Any `<LINEERROR>` or `<LASTMSG>` containing "error" marks failure.

5.3 Domain → voucher mapping

Adamrit object	Tally type	Debit side	Credit side
IPD Bill	Sales	Patient ledger	Per-line income (default "Hospital Income")
Pharmacy Sale	Sales	Patient ledger	"Pharmacy Sales" (mapped)
Advance / Receipt	Receipt	Bank (mapped from paymentMode)	Patient ledger
Payment Voucher	Payment	Per-line debit ledgers	Account/Cash ledger
ESIC Bill	Sales	"ESIC Receivables"	"ESIC Income - {serviceType}"
Insurance Bill (TPA)	Sales	Insurance receivables + patient share	Income ledgers
Insurance Settlement	Receipt	Bank + TDS Receivable + Disallowance	Insurance receivables (gross)

The **insurance settlement** is a notable multi-leg entry: Dr Bank (amount) + optional Dr "TDS Receivable" + optional Dr "Insurance Disallowance"; Cr insurance receivables for the gross (amount + TDS + disallowance). The insurer ledger itself is resolved per-company from `tally_ledger_mapping` where `adamrit_entity_type='insurance'`.

5.4 Retry queue & reliability (src/lib/tally-retry-processor.ts, 109 lines)

`processRetryQueue()` (:9-86) drains `tally_push_queue`:

- Selects up to **10** pending items where `next_retry_at ≤ now`, ordered by `created_at`.
- Loads each item's `company_id` config (multi-company aware).
- Re-POSTs the payload to the proxy. On success → `status='completed'`; on failure → `retry_count+1`, **linear backoff** `min(retry_count × 5 min, 60 min)`.
- After `retry_count ≥ max_retries` (5) → `status='failed_permanent'`.

Trigger: `TallyRetryQueue.tsx` runs `processRetryQueue()` on a **5-minute** `setInterval` (:39-45) and via a "Process Queue Now" button. The interval only runs while the component is mounted.

SILENT DATA-LOSS SURFACE

All logging / queue / mirror helpers swallow their own errors (empty `catch {}`) so a logging failure can never break the primary push — but this means a failed `tally_push_queue` insert would **silently drop** the retry record. Combined with the browser-only scheduler, an unattended push failure with a closed tab can sit indefinitely.

6. Reverse Sync — Tally → Adamrit

Reverse sync pulls data back from Tally via **three complementary mechanisms**, all routed through `/api/tally-proxy`. They serve different needs: bulk mirror, receipt-to-visit reconciliation, and on-demand report rendering.

#	Mechanism	Location	Role
A	Server-side Collection export	<code>handleSync()</code> in <code>api/tally-proxy.ts:288-617</code>	Primary bulk pull. TDL <code><COLLECTION></code> + <code><FETCH></code> , bulk-upsert (50/batch) into Plane C tables via service-role key (bypasses RLS).
B	Client-side receipt matcher	<code>src/lib/tally-reverse-sync.ts</code>	Pulls Day-Book receipts (last 30 d), auto-matches them to patient visits , flips <code>visits.payment_status='paid'</code> . Only path that writes back to clinical data.
C	Client XML report service	<code>src/lib/tally-xml-service.ts</code>	Typed pullers (Trial Balance, Balance Sheet, P&L, outstanding, GST). On-demand by <code>TallyReports.tsx</code> ; caches parsed JSON in <code>tally_reports</code> .

6.1 Server-side Collection pulls (`handleSync`, `api/tally-proxy.ts:288-617`)

On entry it inserts a `tally_sync_log` row (`status='started'`); on exit it computes status (`completed/partial/failed/no_data`), updates `records_synced/records_failed/duration_ms`, and bumps `tally_config.last_sync_at`. Pull strategy by action:

action	Tally request	Upsert target	Timeout
ledgers	Collection <code>Ledger</code> Collection	<code>tally_ledgers</code> (50/batch)	120s
groups	Collection <code>Group</code> Collection	<code>tally_groups</code>	120s
stock	Collection <code>StockItem</code> Collection	<code>tally_stock_items</code>	120s
vouchers	Collection (primary) / Day Book (fallback)	<code>tally_vouchers</code> (50/batch)	180s
reports	5 Data exports (TB/BS/P&L/Recv/Pay)	<code>tally_reports</code> (raw XML, append-only)	120s each
gst-r1 / gst-r3b / gst-ledger	GSTR-1 / GSTR-3B / Day Book	<code>tally_gst_data</code> (raw XML)	120s
full	Recurses groups→ledgers→stock→vouchers→reports	—	—

WHY COLLECTION OVER DAY BOOK

The Day Book has been observed to ignore `SVFROMDATE/SVTODOATE` in some TallyPrime versions. The TDL `<COLLECTION>` path is reliable and faster, so it is the primary; Day Book is a fallback. `api/tally-proxy.ts:458-461`

SVCURRENTCOMPANY DELIBERATELY OMITTED

The proxy intentionally drops `<SVCURRENTCOMPANY>` from export requests (:64-68) because Tally silently returns 0 records when the name doesn't match exactly (case, spacing, `&` vs `&`). It relies on Tally's currently-loaded company instead. **Imports do include it.**

6.2 Client-side receipt matcher (src/lib/tally-reverse-sync.ts, 306 lines)

The orchestrator `reverseSync()` (:70-263) is the only component that writes reconciliation results back into the **clinical** schema (`visits`). It runs in four steps:

1. **Pull Day Book receipts (last 30 days)** via a Data export, regex-extract `<VOUCHER>` where `VOUCHERTYPENAME='Receipt'`, sum negative (debit) `AMOUNT` entries.
2. **Match each receipt to a visit** — `matchReceiptToBill()` (:42-68) queries `visits` where `payment_status IN (unpaid,partial,null)` AND `total_amount` within $\pm 10\%$, ordered by date desc, picks the highest token-overlap name match with `score  $\geq$  0.5`.
3. **Flip `visits.payment_status='paid'`** on each match.
4. **Refresh ledger balances** + discover new ledgers via `List of Ledgers`, upsert into `tally_ledgers`.

Logs a single summary row in `tally_sync_log` with `direction='inward'` and an `error_details` payload of `{matched, updated, newLedgers, unmatchedCount}`.

6.3 Reports & caching (src/components/tally/TallyReports.tsx)

`TallyReports.tsx` is the primary consumer of the client XML service. It pulls Trial Balance, Balance Sheet, P&L, Receivables, Payables on demand, parses the XML to structured JSON, and caches each in `tally_reports` via `cacheReport()` (:235-245). On mount, `loadCachedReports()` (:156-233) rehydrates the latest 10 rows — but deliberately **skips rows whose `data.raw` is a string** (the server-side raw-XML caches), so the structured client cache always wins. Outstanding tables render aging buckets 0–30 / 30–60 / 60–90 / 90+.

7. The TallyPrime XML Protocol

TallyPrime's HTTP gateway speaks a request/response format built on an **<ENVELOPE>** XML document. Every operation — creating a ledger, importing a voucher, pulling a trial balance — is one envelope POSTed as **application/xml** to the gateway root. This section documents the exact shapes Adamrit uses.

7.1 Import envelopes (writes) tally-xml-service.ts:200-244 · api/tally-proxy.ts:184-273

Writes use **TALLYREQUEST=Import**. Master imports use **<ID>All Masters</ID>**; voucher imports use **<ID>Vouchers</ID>**.

Master (ledger) import

```
<ENVELOPE>
  <HEADER>
    <VERSION>1</VERSION>
    <TALLYREQUEST>Import</TALLYREQUEST>
    <TYPE>Data</TYPE>
    <ID>All Masters</ID>
  </HEADER>
  <BODY>
    <DESC><STATICVARIABLES>
      <SVCURRENTCOMPANY>{companyName}</SVCURRENTCOMPANY>
    </STATICVARIABLES></DESC>
    <DATA>
      <TALLYMESSAGE xmlns:UDF="TallyUDF">
        <LEDGER NAME="..." ACTION="Create">
          <NAME>...</NAME>
          <PARENT>...</PARENT>
          <OPENINGBALANCE>...</OPENINGBALANCE>
          <ADDRESS.LIST><ADDRESS>...</ADDRESS></ADDRESS.LIST>
          <LEDGERPHONE>...</LEDGERPHONE>
          <PARTYGSTIN>...</PARTYGSTIN>
          <INCOMETAXNUMBER>...</INCOMETAXNUMBER>
        </LEDGER>
      </TALLYMESSAGE>
    </DATA>
  </BODY>
</ENVELOPE>
```


Voucher import (Sales example)

```
<ENVELOPE>
  <HEADER>
    <TALLYREQUEST>Import</TALLYREQUEST>
    <TYPE>Data</TYPE>
    <ID>Vouchers</ID>          <!-- "Vouchers", NOT "All Masters" -->
  </HEADER>
  <BODY>
    <DESC><STATICVARIABLES>
      <SVCURRENTCOMPANY>{companyName}</SVCURRENTCOMPANY>
    </STATICVARIABLES></DESC>
    <DATA>
      <TALLYMESSAGE xmlns:UDF="TallyUDF">
        <VOUCHER VCHTYPE="Sales" ACTION="Create">
          <DATE>20260708</DATE>          <!-- YYYYMMDD, no dashes -->
          <NARRATION>...</NARRATION>
          <VOUCHERTYPENAME>Sales</VOUCHERTYPENAME>
          <PARTYLEDGERNAME>...</PARTYLEDGERNAME>
          <ALLLEDGERENTRIES.LIST>
            <LEDGERNAME>...</LEDGERNAME>
            <ISDEEMEDPOSITIVE>Yes</ISDEEMEDPOSITIVE>    <!-- debit side -->
            <AMOUNT>-1234.56</AMOUNT>                    <!-- negative = debit -->
          </ALLLEDGERENTRIES.LIST>
          <ALLLEDGERENTRIES.LIST>
            <LEDGERNAME>...</LEDGERNAME>
            <ISDEEMEDPOSITIVE>No</ISDEEMEDPOSITIVE>      <!-- credit side -->
            <AMOUNT>1234.56</AMOUNT>
          </ALLLEDGERENTRIES.LIST>
        </VOUCHER>
      </TALLYMESSAGE>
    </DATA>
  </BODY>
</ENVELOPE>
```

7.2 Export / Collection envelopes (reads)

Reads use **TALLYREQUEST=Export**. Two dialects coexist:

Data export (report-style)

```
<ENVELOPE>
  <HEADER>
    <TALLYREQUEST>Export</TALLYREQUEST>
    <TYPE>Data</TYPE>
    <ID>{reportId}</ID>          <!-- "List of Companies", "Day Book", "Trial Balance", ... -->
  </HEADER>
  <BODY><DESC><STATICVARIABLES>
    <SVEXPORTFORMAT>$$SysName:XML</SVEXPORTFORMAT>
    <!-- SVCURRENTCOMPANY intentionally OMITTED by the proxy -->
    <SVFROMDATE>20260401</SVFROMDATE>
    <SVTODATE>20260708</SVTODATE>
  </STATICVARIABLES></DESC></BODY>
</ENVELOPE>
```

Collection export (TDL) — the reliable bulk-pull dialect

```
<ENVELOPE>
  <HEADER>
    <TALLYREQUEST>Export</TALLYREQUEST>
    <TYPE>Collection</TYPE>
    <ID>Ledger Collection</ID>
  </HEADER>
  <BODY><DESC><STATICVARIABLES></STATICVARIABLES>
    <TDL><TDLMESSAGE>
      <COLLECTION NAME="Ledger Collection" ISMODIFY="No">
        <TYPE>Ledger</TYPE>
        <FETCH>NAME,PARENT,OPENINGBALANCE,CLOSINGBALANCE,
          ADDRESS,LEDGERPHONE,EMAIL,PARTYGSTIN,
          INCOMETAXNUMBER,GUID</FETCH>
      </COLLECTION>
    </TDLMESSAGE></TDL>
  </DESC></BODY>
</ENVELOPE>
```

7.3 Sign & date conventions api/tally-proxy.ts:20-22,198 · tally-xml-service.ts:168-170,740

Convention	Rule
Date format	YYYYMMDD (no dashes). <code>formatTallyDate()</code> converts.
Debit amounts	Negative. <code>isDeemedPositive ? -Math.abs(amt) : Math.abs(amt)</code>
Credit amounts	Positive.
ISDEEMEDPOSITIVE	"Yes" = debit side, "No" = credit side.
ACTION attribute	"Create" (new), "Alter" (edit, needs VOUCHERNUMBER), "Delete" (cancel).

MOST COMMON TALLY XML BUG

Mixing up the **ISDEEMEDPOSITIVE** flag and the **AMOUNT** sign. Debit side = **ISDEEMEDPOSITIVE=Yes** **and** negative amount. Getting either wrong produces unbalanced vouchers or reversed postings.

8. Database Schema Reference

The schema evolved across ~15 migration files. The current shape is multi-tenant via `company_id` UUID → `tally_config(id)` on Plane C, and self-referential trees on Planes A and B. This section summarises every significant table. RLS is enabled everywhere but is **fully permissive** (see §11).

8.1 Plane A — operational ledger (migration 20250612230000)

Table	Key columns	Purpose
<code>chart_of_accounts</code>	<code>account_code</code> (UNIQUE), <code>account_type</code> (13-value CHECK), <code>parent_account_id</code> (self-FK), <code>opening_balance</code> , <code>opening_balance_type</code>	Chart of accounts tree
<code>patient_ledgers</code>	<code>patient_id</code> (UNIQUE FK), <code>account_id</code> , <code>current_balance(_type)</code> , <code>credit_limit</code> , <code>credit_days</code>	Per-patient debtor sub-ledger
<code>voucher_types</code>	<code>voucher_type_code</code> (UNIQUE), <code>voucher_category</code> , <code>numbering_method</code> , <code>current_number</code>	Voucher-type catalog + sequence
<code>vouchers</code>	<code>voucher_number</code> , <code>voucher_type_id</code> , <code>total_amount</code> , <code>patient_id</code> , <code>bill_id</code> , <code>status</code> (PENDING/AUTHORISED/CANCELLED), <code>company_id</code>	Transaction header
<code>voucher_entries</code>	<code>account_id</code> , <code>debit_amount</code> , <code>credit_amount</code> , <code>entry_order</code> , <code>cost_centre_id</code> , <code>bill_ref</code> ; CHECK forbids both Dr & Cr > 0	Dr/Cr legs
<code>outstanding_invoices</code>	<code>patient_id</code> , <code>invoice_number/date/due_date</code> , <code>outstanding_amount</code> , <code>aging_bucket</code> , <code>status</code>	Receivables tracking
<code>payment_transactions</code>	<code>payment_mode</code> (8-value), <code>payment_amount</code> , <code>reference_number</code> , <code>status</code>	Inbound patient payments
<code>payment_allocations</code>	<code>payment_transaction_id</code> , <code>outstanding_invoice_id</code> , <code>allocated_amount</code>	Payment→bill settlement (M:N)
<code>aging_snapshots</code>	6 buckets: <code>bucket_0_30</code> ... <code>365_plus</code>	Historical aging
<code>daily_balances</code>	UNIQUE(<code>balance_date</code> , <code>account_id</code>), opening/closing	Trial-balance support
<code>voucher_edit_log</code>	<code>action</code> (CREATED/ALTERED/CANCELLED/DELETED), <code>old_data/new_data</code> JSONB	Tamper-proof audit (trigger-fed)

8.2 Plane B — Tally-style masters (migrations 20260704140000, 20260704160000, 20260705010000, 20260705020000)

Table	Key columns	Purpose
ledger_groups	parent_group_id (self-FK), behaves_like_subledger, is_predefined; 28 Tally groups seeded	Tally group hierarchy
ledgers	code, group_id, nature (Dr/Cr), opening/current_balance, mailing_*, gstin, pan, bank_*	Tally-style ledger master
payment_vouchers	voucher_no (UNIQUE), person_name, amount, purpose, account_ledger_name, narration	Cash handed out (Tally Payment)
payment_voucher_lines	voucher_id, ledger_id → tally_ledgers, amount, line_order	Payment voucher "Particulars"
cost_centres	name (UNIQUE); 8 departments seeded	Departmental allocation

8.3 Plane C — Tally mirror (migrations 20250625000001, 20250626000000, 20260303000000, 20260321100000)

Table	Key columns / uniqueness	Purpose
tally_config	company_name, server_url, is_active, auto_sync_enabled, sync_interval_minutes, metadata JSONB	Tenant root (one row per Tally company)
tally_sync_log	sync_type, direction (inward/outward), status, records_synced/failed, duration_ms, company_id	Every sync run (both directions)
tally_ledgers	UNIQUE(company_id,name), closing_balance, is_mapped, adamrit_entity_id/type	Pulled ledger master + per-ledger mapping
tally_groups	UNIQUE(company_id,name), nature_of_group, is_revenue	Pulled group master
tally_vouchers	UNIQUE(company_id,tally_guid), ledger_entries JSONB[], sync_direction, sync_status	Pulled / mirrored vouchers
tally_stock_items	UNIQUE(company_id,name), closing_balance/value, gst_rate, hsn_code	Pulled stock master
tally_reports	report_type, period_from/to, data JSONB (raw XML or structured); append-only	Cached report snapshots
tally_gst_data	report_type (gstr1/gstr3b/gst_ledger), data JSONB	Cached GST returns
tally_bank_statements	bank_ledger, deposit/withdrawal, matched_voucher_id, match_status	Bank reconciliation bridge
tally_push_queue	push_action, payload JSONB, status (pending/completed/failed_permanent), retry_count, max_retries, next_retry_at	Outward retry queue
tally_ledger_mapping	UNIQUE(company_id, adamrit_entity_type, adamrit_entity_name) → tally_ledger_name, tally_group	System/default ledger mapping
tally_cost_centres	UNIQUE(company_id,name), category, adamrit_department_id	Cost-centre → department mapping
payment_obligation_ledgers	obligation_id, company_id, ledger_id	Cross-company payables link

IDEMPOTENCY

Every upsert relies on (company_id, natural_key) unique constraints: vouchers on (company_id, tally_guid); ledgers/stock/groups/cost-centres on (company_id, name); the mapping table on (company_id, entity_type, entity_name). When a GUID is missing, the proxy synthesises `_${vchType}-${vchNum}-${date}-${idx}` `api/tally-proxy.ts:508`.

9. The Mapping Layer

Three mapping mechanisms coexist — a deliberate layering for default routing, per-ledger binding, and cost-centre allocation.

Mechanism	Storage	Used by
System / default mapping OUTWARD PUSH	<code>tally_ledger_mapping:</code> <code>(adamrit_entity_type,</code> <code>adamrit_entity_name) → tally_ledger_name</code> <code>[+ tally_group]</code>	<code>getLedgerMapping()</code> in <code>tally-auto-push.ts</code> resolves a payment mode (e.g. "UPI") to its Tally ledger ("HDFC Bank") when constructing push vouchers.
Per-ledger mapping OUTWARD + REVERSE	<code>tally_ledgers.is_mapped /</code> <code>adamrit_entity_id / adamrit_entity_type</code>	<code>TallyMapping.tsx</code> UI binds a synced Tally ledger (usually a Sundry Debtor) to a specific patient. <code>autoMap()</code> matches by name containment.
Cost-centre mapping	<code>tally_cost_centres.adamrit_department_id</code>	Maps Tally cost centres to Adamrit wards/departments/doctors.

Default mapping seed (tally-migration-complete.sql:221-251 · 20260303000000:84-101)

Adamrit entity	Type	Tally ledger	Tally group
Cash	payment_mode	Cash	Cash-in-Hand
Card / UPI / NEFT / RTGS / ONLINE / DD / CHEQUE / Bank Transfer	payment_mode	HDFC Bank	Bank Accounts
Insurance	payment_mode	Insurance Receivables	Sundry Debtors
ESIC	payment_mode	ESIC Receivables	Sundry Debtors
CGHS	payment_mode	CGHS Receivables	Sundry Debtors
Hospital Income (default)	service_category	Hospital Income	Direct Incomes
Pharmacy	pharmacy	Pharmacy Sales	Direct Incomes
ICU / General Ward / OPD	department	Hospital Income - ICU / - General Ward / - OPD	Direct Incomes
Star Health, ICICI Lombard, New India...	insurance	(per-insurer receivable ledger)	Sundry Debtors
ESIC Consultation/IPD/OPD/Surgery/Lab/ Pharmacy	esic_income	ESIC Income - {serviceType}	Direct Incomes

Mapping types: `payment_mode` | `service_category` | `pharmacy` | `department` | `insurance` | `esic_income` | `insurance_expense`.

10. Sync Modes, Scheduling & Triggers

This section consolidates *when* syncs run. The recurring theme: scheduling is browser-resident, not server-resident.

Operation	Trigger	Default range	Resident in
Forward push (live)	Event-driven, inline at save	n/a	Business pages → <code>tally-auto-push.ts</code>
Forward retry drain	5-min <code>setInterval</code> + manual button	n/a	<code>TallyRetryQueue.tsx</code> (browser)
Forward manual re-push	Button click	n/a	<code>TallyBillSync.tsx</code> , <code>TallyCreateVoucher.tsx</code>
Reverse full sync	Dashboard "Sync All" or auto-sync	FY-start (Apr 1) → today	<code>TallyDashboard.tsx</code>
Reverse auto-sync	<code>setInterval</code> (15/30/60/120/360 min)	FY-start → today	<code>TallyDashboard.tsx</code> + <code>TallyAutoSync.tsx</code>
Reverse reports	Per-tab "Fetch from Tally"	As-of today / FY range	<code>TallyReports.tsx</code>
Receipt→visit reverse sync	Programmatic <code>reverseSync()</code>	Last 30 days	<code>tally-reverse-sync.ts</code>
Reverse standalone (HTTP)	OS cron / Task Scheduler	Configurable	<code>scripts/tally-direct-sync.js</code>
Reverse standalone (file)	OS cron	n/a (reads export files)	<code>scripts/sync-tally-export.js</code>

PROXY TIMEOUT ENGINEERING

Test-connection: 60s (1 retry). Masters/vouchers: 120–180s. Reports: 120s. Client-side `runSingleSync()` adds a 5-min `AbortController` per type to survive Vercel function limits. For "full" sync the dashboard loops each type as a **separate API call** to avoid hitting the function timeout. `TallyDashboard.tsx`: 328–347

11. Reconciliation & Reporting Surfaces

How synced data is consumed to close the loop between the two systems.

Use case	Source	Mechanism
Bank reconciliation	<code>tally_vouchers</code> (bank-ledger entries) ↔ <code>tally_bank_statements</code>	<code>TallyBankReconciliation.tsx</code> : auto-match by exact amount ($\pm ₹0.01$) + ≤ 3 -day date proximity; manual two-pane link; CSV/manual statement ingest.
Receipt → visit	<code>visits</code> ← Tally Day Book receipts	<code>tally-reverse-sync.ts</code> : fuzzy name + $\pm 10\%$ amount match → <code>payment_status='paid'</code> . Only path that writes back to clinical data.
Outstanding receivables	<code>tally_reports</code> + <code>tally_ledgers</code> (parent $\approx$ Sundry Debtor)	<code>getOutstandingReceivables</code> pulls <code><BILL></code> elements; aging buckets 0-30/30-60/60-90/90+.
GST returns	<code>tally_gst_data</code> + <code>tally_vouchers.ledger_entries</code>	<code>TallyGST.tsx</code> falls back to voucher-derived totals (substring match <code>igst/cgst/sgst</code>) when no cached pull exists.
Trial Balance / BS / P&L	<code>tally_reports</code>	<code>TallyReports.tsx</code> fetches on demand, caches parsed JSON; server-side stores raw XML.
Financial snapshot	<code>tally_ledgers.closing_balance</code>	<code>TallyDashboard.loadStats()</code> buckets by <code>parent_group</code> keywords (cash/bank/sundry debtor/creditor).
Cash/Bank books	<code>tally_vouchers.ledger_entries</code> + opening balances	Running balance computed client-side; realtime Supabase channels merge new vouchers.
Bill aging / director receivables	Plane A <code>aging_snapshots</code> + <code>payment_obligation_ledgers</code>	Cross-company outstanding view for directors.

12. Error Handling, Idempotency & Security

Reliability characteristics

- **Per-push try/catch** — every `pushXxx` wraps the fetch, logs to `tally_sync_log`, enqueues on failure.
- **Silent skip when inactive** — if `isTallyActive()` is false, most pushes return immediately (except `pushLedgerToTally`, which enqueues regardless).
- **Mirror-write idempotency** — dedup by `(company_id, voucher_number)` count-check.
- **Linear retry backoff** — `min(retryCount × 5 min, 60 min)`, max 5 retries → `failed_permanent`.
- **Reachability errors** surfaced as human-readable messages (Tally running / port enabled / firewall).

Security posture

PERMISSIVE RLS — DOCUMENTED, BY DESIGN

Every Plane A and Plane C table enables RLS but with a fully permissive policy (`USING (true) WITH CHECK (true)` for `anon`, `auth.role() = 'authenticated'` FOR ALL otherwise). There is **no tenant or role isolation at the database layer**. Multi-company separation is enforced only in application code by filtering `company_id`. The service-role proxy bypasses RLS entirely.

CREDENTIALS

TallyPrime's HTTP gateway is **unauthenticated** (relies on network-level access control). The proxy's only secret is `SUPABASE_SERVICE_ROLE_KEY` (server-side env). The Tally `serverUrl` is stored in plaintext in `tally_config`. The proxy sets `Access-Control-Allow-Origin: *` — acceptable because it is only an XML relay, but worth noting.

13. Known Gaps & Architectural Caveats

1. NO SERVER-SIDE WORKER

Both the forward retry drain and the reverse auto-sync run as browser `setInterval` loops. They only progress while a user has the relevant Tally page open. An unattended failure can sit indefinitely, and closing the tab silently halts scheduled sync.

2. ONLY RECEIPTS ARE AUTOMATED

Money-in events auto-post. Revenue recognition, expenses, purchases, payroll, ESIC, and insurance require manual Voucher Entry. Undisciplined data entry produces incomplete books.

3. TWO COEXISTING CONFIG SCHEMAS

`tally_server_url/tally_port/sync_frequency` (legacy, used by `useTallyIntegration`) vs `server_url/is_active/metadata` (active, used by auto-push + proxy). Reconciled by `20250626000001_fix_tally_config_columns.sql`.

4. DEAD HOOK/SERVICE LAYER

`src/services/tallyIntegration.ts` + `src/hooks/useTallyIntegration.ts` reference methods that do not exist (`initialize`, `syncLedgers`, `exportData...`) and would throw at runtime. They are an abandoned earlier architecture; the live path is `tally-auto-push.ts` + `api/tally-proxy.ts`.

5. PLANE B TABLES CREATED OUT-OF-BAND

`ledgers` and `ledger_groups` base DDL has no tracked migration. A from-scratch migration replay would **not** create them.

6. TRIGGER FUNCTION REDEFINED ~10 TIMES

`create_receipt_voucher_for_payment()` has been `CREATE OR REPLACED` repeatedly; only `20260704200000` is authoritative. Earlier versions routed all non-cash to Cash, credited generic `4000`, and created zero-amount vouchers.

7. BANK ROUTING PARTLY KEYWORD-BASED

Parsing `remarks` for "sbi"/"saraswat" is backward-compat. The preferred path is the explicit `bank_account_id` column.

14. Appendix — File & Code Reference Index

Forward sync

File	Role
<code>api/tally-proxy.ts</code>	Vercel proxy. The only HTTP-to-Tally component. <code>handlePush</code> = forward entry.
<code>src/lib/tally-auto-push.ts</code>	Primary forward-sync layer. Named push functions wired into business flows.
<code>src/lib/tally-retry-processor.ts</code>	Drains <code>tally_push_queue</code> with linear backoff.
<code>src/lib/tally-proxy.ts</code>	Thin client wrapper over <code>/api/tally-proxy</code> for UI-driven pushes.
<code>src/lib/tally-xml-service.ts</code>	Typed client-side XML builder (import/export). Parallel to auto-push.
<code>src/components/tally/TallyBillSync.tsx</code>	Manual outbound push dashboard.
<code>src/components/tally/TallyCreateVoucher.tsx</code>	Manual create-voucher form (3-step push + mirror).
<code>src/components/tally/TallyVouchers.tsx</code>	Voucher browser + alter/cancel against Tally.

Reverse sync

File	Role
<code>api/tally-proxy.ts</code> <code>handleSync</code>	Primary bulk pull engine (TDL Collections).
<code>src/lib/tally-reverse-sync.ts</code>	Receipt matcher; writes back to <code>visits</code> .
<code>src/components/tally/TallyDashboard.tsx</code>	Control tower + auto-sync scheduler.
<code>src/components/tally/TallyAutoSync.tsx</code>	30-s-poll scheduler (1–24h intervals).
<code>src/components/tally/TallyReports.tsx</code>	TB/BS/P&L/Receivables/Payables consumer + cache.
<code>src/components/tally/TallyBankReconciliation.tsx</code>	Bank recon (auto + manual match).
<code>src/components/tally/TallyGST.tsx</code>	GSTR-1/3B + GST ledger (cached or derived).
<code>src/components/tally/TallyMapping.tsx</code>	Ledger & cost-centre mapping UI.
<code>scripts/tally-direct-sync.js</code>	Standalone HTTP pull script (cron).
<code>scripts/sync-tally-export.js</code>	File-based pull script (cron).

Accounting module

File	Role
<code>src/components/accounting/AccountingPage.tsx</code>	Module shell + Tally chrome + 32 destinations.
<code>src/components/accounting/GatewayOfTally.tsx</code>	Pixel-faithful Gateway of Tally home.
<code>src/components/accounting/TallyLedgerCreation.tsx</code>	Ledger creation/alteration (Plane B ledgers).
<code>src/components/accounting/TallyImportExport.tsx</code>	File-based XML bridge (masters + vouchers).
<code>src/hooks/useAccountingData.ts</code>	React Query hooks for Plane A.
<code>src/types/accounting.ts</code>	TypeScript domain model.
<code>complete_accounting_setup.sql</code>	Standalone bootstrap of Plane A.

Key migrations

Migration	Creates / changes
<code>20250612230000_create_accounting_tables.sql</code>	Plane A: 10 tables, balance trigger, RLS
<code>20250613000000_insert_accounting_default_data.sql</code>	Canonical CoA + voucher types
<code>20250614000000_fix_accounting_tables_rls_for_anonymous.sql</code>	Permissive anon RLS
<code>20250615000000_add_foreign_keys_to_accounting_tables.sql</code>	User FK columns + indexes
<code>20250625000001_create_tally_integration_tables.sql</code>	Early tally tables + import/export logs
<code>20250626000000_create_tally_sync_tables.sql</code>	Plane C: tally_sync_log/ledgers/groups/vouchers/stock/reports/gst
<code>20250626000001_fix_tally_config_columns.sql</code>	Reconciles config schemas
<code>20260303000000_create_tally_missing_tables.sql</code>	mapping, push_queue, cost_centres + mapping cols
<code>20260522120001_pharmacy_bridge_dispense_sync.sql</code>	Dispense state sync triggers (upstream of accounting)
<code>20260704140000_tally_style_payment_voucher.sql</code>	payment_voucher_lines
<code>20260704160000_tally_ledger_groups.sql</code>	ledger_groups Tally cols + 28 seeded groups
<code>20260704200000_receipt_voucher_proper_ledgers.sql</code>	Authoritative receipt-voucher trigger

End of document · Adamrit ↔ Tally Sync Architecture & Accounting Module · Generated from source code analysis. All technical claims anchored to **file:line** references in the Adamrit repository.